Customized Posit Adders and Multipliers using the FloPoCo Core Generator

Raul Murillo, Alberto A. Del Barrio and Guillermo Botella
Department of Computer Architecture and Automation
Complutense University of Madrid
Madrid, Spain
Email: {ramuri01, abarriog, gbotella}@ucm.es

***Abstract*—The posit number system, which is proposed as a replacement of IEEE floating-point numbers, is in the spotlight of Arithmetic research due to the recent breakthroughs. This format claims to provide more accurate results with the same bitwidth than standard floating point, but the run-time variability during the detection of the posit fields involves a hardware design challenge. In this work, we propose parameterized designs for multiple posit functional units, including addition and multiplication, and integrate them as templates of the FloPoCo framework. The integration of the proposed algorithms within FloPoCo can provide synthesizable VHDL code for posit arithmetic of any possible configuration $\langle n, es\rangle$. Experiments show an improvement in terms of area and energy with respect to state-of-the-art works up to 35.9% and 30.8%, respectively.**

I. Introduction

The posit number system, proposed by John L. Gustafson in [1] as an alternative to the ubiquitous standard for floating-point arithmetic (IEEE 754), is drawing attention due to the multiple benefits it promises, such as better dynamic range and accuracy over same bit width, better computations closure or consistency between machines than with conventional floating point. Recent researches have shown posits to be very promising in many different areas as numerical analysis [2], HPC [3] or Deep Learning [4]. However, posits are still in development and there is still some debate about their improvement [2].

The numerical value of a posit number X, whose bits are distributed as shown in Fig. 1, is given by (1):

$$X = (-1)^s * (useed)^k * 2^e * 1.f \ , \tag{1}$$

$$k = -x_{n-2} + \sum_{i=n-2}^{x_i \neq x_{n-2}} (-1)^{1-x_i} \ , \tag{2}$$

where $useed = 2^{2^{es}}$, k is the *regime* value, e is the unsigned exponent (if $es > 0$), and f is the mantissa of the number without the implicit one. The scaling factor of a posit number will then be determined by both the regime (k) and the exponent value (e). Concerning to the format layout, the main differences with floating point are the existence of the regime field and the utilization of an unsigned and unbiased exponent, if there exists such exponent field. The regime is a sequence of bits with the same value (r) finished with the negation of such value ($\overline{r}$). Provided that $X = x_{n-1}x_{n-2}...x_1x_0$, this regime can be expressed as Equation (2) shows.

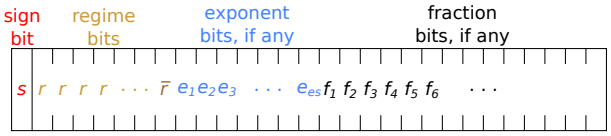

Fig. 1. Layout of an $\langle n, es\rangle$ posit number.

In other words, the regime basically counts the number of occurrences of the bit labelled as r in Fig. 1. If $r = 1$ then the regime is the number of 1's minus 1, while if $r = 0$, the regime is the negative value of the number of 0's. For instance, if the regime is 4-bits wide, the value `1110` would be interpreted as $k = 2$, while the value `0001` would be $k = -3$. Hence, detecting the leading 1's and 0's is critical for performing this step [5]–[7]. However, the main difficulty to detect the regime, and consequently unpack the posit, is that its length varies dynamically. In numbers close to zero, there will be many fraction bits and few regime bits and, in large numbers there will be many regime bits and fewer fraction bits, but there is no fixed amount of bits.

This variation in the configurations given by the posit numbers give is a brilliant opportunity for research into the design and development of efficient units able to implement operations among posits. According to [8], a simple way of obtaining similar dynamic range as IEEE floats consists of increasing the es value of posits format every time the total number of bits is doubled. Therefore, common posit formats are: Posit$\langle 8, 0\rangle$ (w.r.t. minifloat [9]), Posit$\langle 16, 1\rangle$ (w.r.t. half precision) and Posit$\langle 32, 2\rangle$ (w.r.t. single precision). Nevertheless, the aforementioned variability of posit configurations opens the door to look for functional units providing the best trade-offs [10], [11]. Therefore, to implement posit functional units, it is important to provide a standardized architecture and a generic flow. FloPoCo (Floating-Point Cores, but not only) is an open-source C++ framework for the generation of arithmetic datapaths which provides a command-line interface that inputs operator specifications and outputs synthesizable VHDL specially suited for Field-Programmable Gate Arrays (FPGAs) [12]. Hence, in this paper we propose designs for adder/subtractor and multiplier posit functional units and their implementation in the latest version of FloPoCo framework, allowing this way to automatically generate synthesizable posit

978-1-7281-3320-1/20/$31.00 ©2020 IEEE
Authorized licensed use limited to: Politecnico di Torino. Downloaded on March 24,2025 at 15:26:55 UTC from IEEE Xplore. Restrictions apply.

operators with any posit configuration (including $es = 0$), but leveraging the multiple FloPoCo capabilities.

The rest of the paper is organized as follows: Section II describes the state of the art regarding posit hardware research and implementations; Section III presents the design of posit adder/subtractor and multiplier operators; Section IV evaluates the synthesis results of FloPoCo generated operators, and finally, Section V gives the final remarks of this work.

II. Related Work

Since the posit number system was introduced in 2017, the interest on a hardware implementation for this format has increased rapidly, and different approaches have been proposed so far.

The posit arithmetic generator proposed in [13], [14] includes parameterized designs of floating-point to posit and vice-versa converters, and posit addition/subtraction and multiplication units. Another parameterized design for posit arithmetic unit that includes adders and multipliers is presented in [15]. This work improves the previous one by using only a leading zero detector for decoding the regime—simplifying the posit data extraction flow and reducing area—, but requires to invert the number if the regime is a sequence of `1`s. Other works that provide parameterized designs [16], [17] also take this approach for posit decoding. The work in [16] presents a C++ template compliant with Intel OpenCL SDK that automatically generates and pipelines posit operations according to bitwidth and exponent size constraints. Authors in [17] improve their prior works [13], [14] by employing just one leading one detector and providing support for posit division. Although these works seem totally parameterized, either do not support for the case $es = 0$, or do no provide results for such cases. This configuration is important because allows to use more fraction bits, and specially useful for low-precision arithmetic; in fact, the Posit$\langle 8, 0\rangle$ configuration seems very promising in the Deep Learning scenario, as it provides an extremely fast approach for the sigmoid function [1]. Authors in [18] implement a generic posit multiplication algorithm into the open-source FloPoCo framework (version 4.1.2), allowing to generate parameterized and combinational/pipelined operators in VHDL, and also including support for $es = 0$.

Finally, for the sake of completeness, it must be noted that since the posit standard [19] includes fused operations such as the fused dot product, and due to the importance of this operation for matrix calculus, some research and development for this kind of implementations has been done. Different fused multiply-add (FMA) units for posits are presented in [20]–[25]. They make use of the *quire* register [19] to accumulate the partial additions that are involved in the dot product, so the result is rounded only after the whole computation.

This paper proposes parameterizable algorithms for posit addition/subtraction and multiplication of two posit numbers, as well as the posit decoding and encoding—which are common stages for all the posit arithmetic operations. These algorithms have been implemented in FloPoCo, version 5.0.

III. Posit Functional Units

The proposed posit arithmetic units are discussed in this section. At hardware level, posits were designed to be easy to compute, i.e., to have a circuitry similar to the existing floating point. The main encoding difference between float and posit formats is the fact that the second one includes a run-time varying scaling component—the regime and the available exponent bits.

A. Posit data extraction

Due to the variable-length fields of the posit format, the operand fields decoding flow slightly differs from the analogous stage for IEEE-754 numbers. The proposed posit decoding process (Algorithm 1) first extracts the sign bit (MSB) and detects zero and NaR special cases performing `OR reduce` on the remaining $n - 1$ bits. In case the input posit value is negative, the algorithm takes its absolute value via 2's complement, simplifying the data extraction process and future operations such as addition of two posits. Then, the regime is computed and shifted-out and, afterwards, the exponent bits (if any) and fraction fields are effortlessly extracted. The main difference between the proposed posit decoder and previous implementations is the use of a leading zero or one counter with integrated shifter (`LZOC+Shift`) instead of using separate detectors for decoding both positive and negative regimes [13], [14], [24] or using a single detector but always modifying the operand to deal with the same regime sign [15]–[18] and then shifting out the regime. This approach simplifies the datapath substantially. Furthermore, the computed absolute value of the posit is not discarded, since it might be useful for operations such as the addition of two posits.

Algorithm 1 Posit data extraction

Require: $X \in \text{Posit}\langle n, es\rangle$
Ensure: $X = (-1)^{sign} * (2^{2^{es}})^{reg} * 2^{exp} * frac$

1: $sign \leftarrow X[n-1]$
2: $nzero \leftarrow \texttt{OR}(X[n-2:0])$ // Reduction OR
3: $z \leftarrow \neg(sign \vee nzero)$
4: $inf \leftarrow sign \wedge \neg(nzero)$
5: $twos \leftarrow (\{n-1\{sign\}\} \oplus in[n-2:0]) + sign$
6: $rc \leftarrow twos[n-2]$ // Regime sign
7: $shifted, count \leftarrow \texttt{LZOC+Shift}(twos, rc)$
8: $exp \leftarrow shifted[MSB-2 : MSB-es-1]$
9: $frac \leftarrow \{nzero, shifted[MSB-es-2 : MSB-es-frac_{size}-1]\}$
10: **if** $rc == 0$ **then**
11: $\quad reg \leftarrow -\{0, count\}$
12: **else**
13: $\quad reg \leftarrow \{0, count-1\}$
14: **end if**
15: **return** $sign, reg, exp, frac, z, inf, twos$

Authorized licensed use limited to: Politecnico di Torino. Downloaded on March 24,2025 at 15:26:55 UTC from IEEE Xplore. Restrictions apply.

B. Posit data encoding and rounding

The pseudocode of posit output process is presented in Algorithm 2. Firstly, the scaling factor is split into exponent and regime. The latter is converted to positive to detect the overflow exception and aid the final regime encoding, which is done by right-shifting the exponent and fraction—according to the sign of regime. Posits, same as IEEE 754 floats, follow round-to-nearest-even scheme. Therefore, the LSB, Guard (G), Round (R) and Sticky (S) must be computed to perform a correct unbiased rounding. Finally, the result is obtained considering the final sign bit and special cases.

Algorithm 2 Posit data encoding

Require: $sign, sf, norm_f, inf, z$ fields of $R \in \text{Posit}\langle n, es\rangle$
Ensure: $R \in \text{Posit}\langle n, es\rangle$ and is correctly rounded

$exp_F \leftarrow sf[es-1:0]$
$reg_F \leftarrow sf[MSB-1:es]$
$rc \leftarrow sf[MSB-1]$
if $rc == 1$ **then**
  $reg_F \leftarrow -reg_F$ // Get regime's absolute value
end if
if $reg_F > n-2$ **then**
  $reg_F \leftarrow n-2$ // Regime overflow
end if
if $rc == 1$ **then**
  $in_{shift} \leftarrow \{0, 1, exp_F, norm_f\}$
  $offset \leftarrow reg_F - 1$
else
  $in_{shift} \leftarrow \{1, 0, exp_F, norm_f\}$
  $offset \leftarrow reg_F$
end if
$ans_{shf} \leftarrow in_{shift} \gg (offset, \neg rc)$
$ans_{tmp} \leftarrow ans_{shf}[MSB-1 : MSB-(n-1)]$
$LSB, G, R \leftarrow ans_{shf}[MSB-(n-1) : MSB-(n+1)]$
$S \leftarrow \texttt{OR}(ans_{shf}[MSB-(n+2):0])$
$round \leftarrow G \wedge (LSB \vee R \vee S)$
if $sign == 1$ **then**
  $result \leftarrow \{1, -(ans_{tmp} + round)\}$
else
  $result \leftarrow \{0, (ans_{tmp} + round)\}$
end if
if $inf == 1$ **then**
  $R \leftarrow \{1, \{n-1\{0\}\}\}$
else if $z == 1$ **then**
  $R \leftarrow \{n\{0\}\}$
else
  $R \leftarrow result$
end if
return R

C. Posit adder/subtractor core

The addition of two posits is similar to how it is performed in the IEEE-754 format (Algorithm 3). First, the larger and smaller operands are detected—note that posit comparison can be done as integers [1], [8]. The fraction of the smaller operand is right shifted (according to the difference of scaling factors of both operands) prior to being added/subtracted to the largest one. Then, the leading zeros of the resulting fraction are shifted out to get a normalized field. The final sign and scaling factor are taken from the larger operand, subtracting the number of leading zeros from the fraction and adding one in case it is overflowed; therefore, the regime overflow may only occur when $es = 0$, and logic from encoding stage can be reduced under this consideration. Note that since operands are previously decoded, subtraction operation just differs on flipping a sign bit before performing the addition flow.

Algorithm 3 Posit addition

Require: $X, Y \in \text{Posit}\langle n, es\rangle$
Ensure: $R = X + Y$

$sign_X, k_X, e_X, f_X, inf_X, abs_X \leftarrow \texttt{Decoder}(X)$
$sign_Y, k_Y, e_Y, f_Y, inf_Y, abs_Y \leftarrow \texttt{Decoder}(Y)$
$OP \leftarrow sign_X \oplus sign_Y$
$inf \leftarrow inf_X \vee inf_Y$
$sf_X \leftarrow \{k_X, e_X\}$
$sf_Y \leftarrow \{k_Y, e_Y\}$
if $abs_X > abs_Y$ **then**
  $sign_L, sf_L, f_L \leftarrow sign_X, sf_X, f_X$
  $sf_S, f_S \leftarrow sf_Y, f_Y$
else
  $sign_L, sf_L, f_L \leftarrow sign_Y, sf_Y, f_Y$
  $sf_S, f_S \leftarrow sf_X, f_X$
end if
$offset \leftarrow |sf_X - sf_Y|$
$f_S \leftarrow f_S \gg offset$
if $OP == 1$ **then**
  $f_{add} \leftarrow f_L - f_S$
else
  $f_{add} \leftarrow f_L + f_S$
end if
$ovf_f \leftarrow f_{add}[MSB-1]$
$norm_f, lzCount \leftarrow \texttt{LZC+Shift}(f_{add})$ // Normalize
$sf_{add} \leftarrow sf_L + ovf_f - lzCount$
$R \leftarrow \texttt{Encoder}(sign_L, sf_{add}, norm_f, inf)$
return R

D. Posit multiplier core

The posit multiplication process (Algorithm 4) starts by detecting the output sign and corner cases. Computing the resulting fraction requires an integer multiplier whose inputs have a bitwidth of $frac_{size} = n - es - 2$ bits. The resulting scaling factor is obtained by adding both operand scales, plus the possible fraction overflow—in this case, this field has to be normalized shifting one bit to the right.

IV. Evaluation

All the posit arithmetic designs presented in this paper have been compared with the reference SoftPosit library[1]. Posit

[1] gitlab.com/cerlane/SoftPosit

Authorized licensed use limited to: Politecnico di Torino. Downloaded on March 24,2025 at 15:26:55 UTC from IEEE Xplore. Restrictions apply.

Algorithm 4 Posit multiplication

Require: $X, Y \in \text{Posit}\langle n, es\rangle$
Ensure: $R = X * Y$

$sign_X, k_X, e_X, f_X, z_X, inf_X \leftarrow \texttt{Decoder}(X)$
$sign_Y, k_Y, e_Y, f_Y, z_Y, inf_Y \leftarrow \texttt{Decoder}(Y)$
$sign \leftarrow sign_X \oplus sign_Y$
$z \leftarrow z_X \vee z_Y$
$inf \leftarrow inf_X \vee inf_Y$
$sf_X \leftarrow \{k_X, e_X\}$
$sf_Y \leftarrow \{k_Y, e_Y\}$
$f_{mult} \leftarrow f_X * f_Y$
$ovf_f \leftarrow f_{mult}[MSB - 1]$
if $ovf_f == 1$ **then**
  $norm_f \leftarrow \{0, f_{mult}\}$
else
  $norm_f \leftarrow \{f_{mult}, 0\}$
end if
$sf_{mult} \leftarrow sf_X + sf_Y + ovf_f$
$R \leftarrow \texttt{Encoder}(sign, sf_{mult}, norm_f, inf, z)$
return R

operators with standard configurations—$\langle 8, 0\rangle$, $\langle 16, 1\rangle$ and $\langle 32, 2\rangle$—have been generated with FloPoCo and synthesized using Synopsys Design Compiler with a 65 nm target-library and without placing any timing constraint. The delay, area, power and energy of different posit functional units have been measured in the following experiments.

We have compared our results with the state-of-the-art designs proposed at [17], [18], which are open-source and publicly available. Unfortunately, the source code from [17] has limitations for generating posit arithmetic units with $es = 0$, and [18] only presents the design of a posit multiplier. We therefore compare our design with the analogous from these two previous works. It must be noted that FloPoCo allows to automatically pipeline the operators with the selected frequency, however, since work from [17] does not provide pipelined designs for every posit configuration, we only generate combinational operators in order to perform fair comparisons.

Table I shows the synthesis results for posit adders. The proposed designs reduce area and power consumption in more than 32% and 26%, respectively. On the other hand, the proposed unit increases the delay around 18% with respect to [17]. However, energy consumption—which is proportional to both delay and power—is lower in the posit adders introduced by this work.

Regarding the posit multiplication units, synthesis results are shown in Table II. Again, with respect to the operators from [17], area, power and energy decrease (around 27%, 13.5% and 8%, respectively), while delay slightly increases, 9.5% in the worst case. In addition, the proposed multipliers improve the synthesis results in all the parameters in comparison with [18].

TABLE I
POSIT ADDERS SYNTHESIS RESULTS.

	Posit $\langle n, es\rangle$	Area (μm^2)	Delay (ns)	Power (μW)	Energy (pJ)
[17]	$\langle 16, 1\rangle$	3228.48	5.34	1637.6	8.74
	$\langle 32, 2\rangle$	7615.08	7.94	3828.3	30.4
This work	$\langle 8, 0\rangle$	1038.6	3.9	489.5	1.91
	$\langle 16, 1\rangle$	2176.92	6.23	1133.1	7.06
	$\langle 32, 2\rangle$	4880.88	9.48	2811.1	26.65

TABLE II
POSIT MULTIPLIERS SYNTHESIS RESULTS.

	Posit $\langle n, es\rangle$	Area (μm^2)	Delay (ns)	Power (μW)	Energy (pJ)
[17]	$\langle 16, 1\rangle$	4955.76	5.15	3036.6	15.64
	$\langle 32, 2\rangle$	15106.32	8.54	13027	111.25
[18]	$\langle 8, 0\rangle$	1271	3.36	612.4	2.06
	$\langle 16, 1\rangle$	3865	6.2	2609.6	16.18
	$\langle 32, 2\rangle$	21894	9.6	13053.3	125.31
This work	$\langle 8, 0\rangle$	1032.48	2.98	558.4	1.66
	$\langle 16, 1\rangle$	3321.72	5.64	2470.9	13.94
	$\langle 32, 2\rangle$	11924.64	8.87	11926	105.78

V. CONCLUSIONS

In this work, two parameterized algorithms for performing addition and multiplication of two posit numbers have been presented. Such algorithms have been integrated into the FloPoCo framework in order to obtain synthesizable VHDL code for any bitwidth (n) and exponent size (es) configuration. Results show an improvement up to 35.9% in terms of area with respect to the state-of-the-art works. Furthermore, our proposed units are more energy efficient.

In order to complement this work, future researching lines involve adding support for missing operations, including the fused operations, which make use of the quire register.

ACKNOWLEDGMENT

This paper has been supported by the CM under grant S2018/TCS-4423, the EU (FEDER) and the Spanish MINECO under grant RTI2018-093684-B-I00 and by Banco Santander under grant PR26/16-20B-1. We would like to thank Prof. Florent de Dinechin for the invitation to collaborate on the FloPoCo project.

REFERENCES

[1] J. L. Gustafson and I. T. Yonemoto, "Beating floating point at its own game: Posit arithmetic," *Supercomputing Frontiers and Innovations*, vol. 4, no. 2, 06 2017.
[2] F. de Dinechin, L. Forget, J.-M. Muller, and Y. Uguen, "Posits: The good, the bad and the ugly," in *Proceedings of the Conference for Next Generation Arithmetic 2019 on - CoNGA'19*. ACM Press, 2019.
[3] L. van Dam, "Enabling high performance posit arithmetic applications using hardware acceleration," Master's thesis, Delft University of Technology, the Netherlands, 2018.
[4] J. Johnson, "Rethinking floating point for deep learning," *arXiv preprint arXiv:1811.01721*, 2018.

Authorized licensed use limited to: Politecnico di Torino. Downloaded on March 24,2025 at 15:26:55 UTC from IEEE Xplore. Restrictions apply.

[5] M. S. Kim, A. A. D. Barrio, R. Hermida, and N. Bagherzadeh, "Low-power implementation of mitchell's approximate logarithmic multiplication for convolutional neural networks," in *2018 23rd Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 1 2018, pp. 617–622.

[6] M. S. Kim, A. A. D. Barrio, L. T. Oliveira, R. Hermida, and N. Bagherzadeh, "Efficient mitchell's approximate log multipliers for convolutional neural networks," *IEEE Transactions on Computers*, vol. 68, no. 5, pp. 660–675, 05 2019.

[7] L. T. Oliveira, M. S. Kim, A. A. D. Barrio, N. Bagherzadeh, and R. Menotti, "Design of power-efficient FPGA convolutional cores with approximate log multiplier," in *European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*, 2019, pp. 203–208.

[8] J. L. Gustafson. Posit arithmetic. [Online]. Available: https://posithub.org/docs/Posits4.pdf

[9] R. Munafo. Survey of floating-point formats. [Online]. Available: http://www.mrob.com/pub/math/floatformats.html

[10] A. A. Del Barrio, R. Hermida, and S. Ogrenci-Memik, "A combined arithmetic-high-level synthesis solution to deploy partial carry-save radix-8 booth multipliers in datapaths," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 66, no. 2, pp. 742–755, 02 2019.

[11] A. A. D. Barrio and R. Hermida, "A slack-based approach to efficiently deploy radix 8 booth multipliers," in *2017 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, mar 2017, pp. 1153–1158.

[12] F. de Dinechin and B. Pasca, "Designing custom arithmetic data paths with FloPoCo," *IEEE Design & Test of Computers*, vol. 28, no. 4, pp. 18–27, 07 2011.

[13] M. K. Jaiswal and H. K.-H. So, "Universal number posit arithmetic generator on FPGA," in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, mar 2018, pp. 1159–1162.

[14] M. K. Jaiswal and H. K.-H. So, "Architecture generator for type-3 unum posit adder/subtractor," in *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2018, pp. 1–5.

[15] R. Chaurasiya *et al.*, "Parameterized posit arithmetic hardware generator," in *2018 IEEE 36th International Conference on Computer Design (ICCD)*. IEEE, oct 2018, pp. 334–341.

[16] A. Podobas and S. Matsuoka, "Hardware implementation of POSITs and their application in FPGAs," in *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, may 2018, pp. 138–145.

[17] M. K. Jaiswal and H. K.-H. So, "PACoGen: A hardware posit arithmetic core generator," *IEEE Access*, vol. 7, pp. 74 586–74 601, 2019.

[18] R. M. Montero, A. A. Del Barrio, and G. Botella, "Template-based posit multiplication for training and inferring in neural networks," *arXiv preprint arXiv:1907.04091*, 2019.

[19] Posit Working Group. Posit standard documentation. [Online]. Available: https://posithub.org/docs/posit_standard.pdf

[20] J. Chen, Z. Al-Ars, and H. P. Hofstee, "A matrix-multiply unit for posits in reconfigurable logic leveraging (open)CAPI," in *Proceedings of the Conference for Next Generation Arithmetic on - CoNGA'18*. ACM Press, 2018, pp. 1:1–1:5.

[21] L. van Dam, J. Peltenburg, Z. Al-Ars, and H. P. Hofstee, "An accelerator for posit arithmetic targeting posit level 1 BLAS routines and pair-HMM," in *Proceedings of the Conference for Next Generation Arithmetic 2019 on - CoNGA'19*. ACM Press, 2019.

[22] Z. Carmichael, H. F. Langroudi, C. Khazanov, J. Lillie, J. L. Gustafson, and D. Kudithipudi, "Deep positron: A deep neural network using the posit number system," in *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, mar 2019, pp. 1421–1426.

[23] Z. Carmichael, H. F. Langroudi, C. Khazanov, J. Lillie, J. L. Gustafson, and D. Kudithipudi, "Performance-efficiency trade-off of low-precision numerical formats in deep neural networks," in *Proceedings of the Conference for Next Generation Arithmetic 2019 on - CoNGA'19*. ACM Press, 2019, pp. 3:1–3:9.

[24] H. Zhang, J. He, and S.-B. Ko, "Efficient posit multiply-accumulate unit generator for deep learning applications," in *2019 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, may 2019, pp. 1–5.

[25] Y. Uguen, L. Forget, and F. de Dinechin, "Evaluating the hardware cost of the posit number system," in *2019 29th International Conference on Field-Programmable Logic and Applications (FPL)*. Barcelona, Spain: IEEE, 09 2019, pp. 1–8.

Authorized licensed use limited to: Politecnico di Torino. Downloaded on March 24,2025 at 15:26:55 UTC from IEEE Xplore. Restrictions apply.